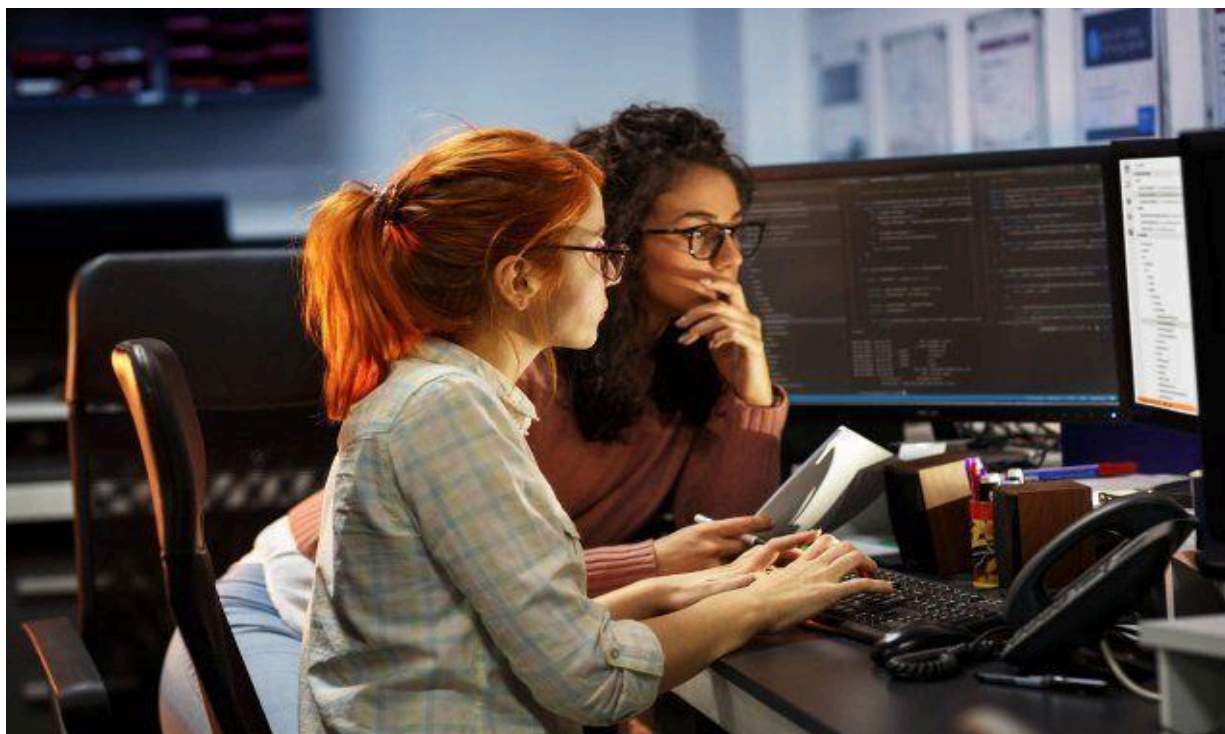




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 4

Tema: Persistencia, Archivos y Concurrencia Moderna

SEMANA 14

Autor:

Introducción:

Bienvenidos al material de lectura #4 de la Unidad #4 de la materia Programación Orientada a Objetos.

En esta unidad, exploraremos un mecanismo avanzado y fundamental de la Persistencia de Datos: la Serialización de Objetos. Aprenderás cómo las aplicaciones Java pueden ir más allá del manejo de texto y datos primitivos para guardar y recuperar objetos completos de la memoria, permitiendo que su estado y estructura sobrevivan la finalización del programa.

Este documento te proporcionará una base técnica sólida, cubriendo el concepto esencial de la Serialización, la diferencia entre un objeto en memoria y sus datos persistentes, y las clases específicas de Java (ObjectOutputStream y ObjectOutputStream) diseñadas para realizar este proceso. Al dominar la Lectura y escritura de objetos, comprenderás la base de muchas tecnologías modernas de comunicación y almacenamiento de datos.

Al finalizar, adquirirás las habilidades esenciales para el desarrollo de cualquier aplicación que necesite almacenar o intercambiar estructuras de datos complejas de manera simple y eficiente. Te animamos a experimentar activamente con los ejemplos de código para consolidar tu comprensión de estas técnicas vitales en la programación orientada a objetos.

Semana 14

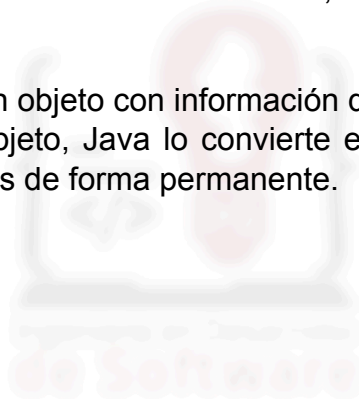
4.2.4. Lectura y escritura de objetos.

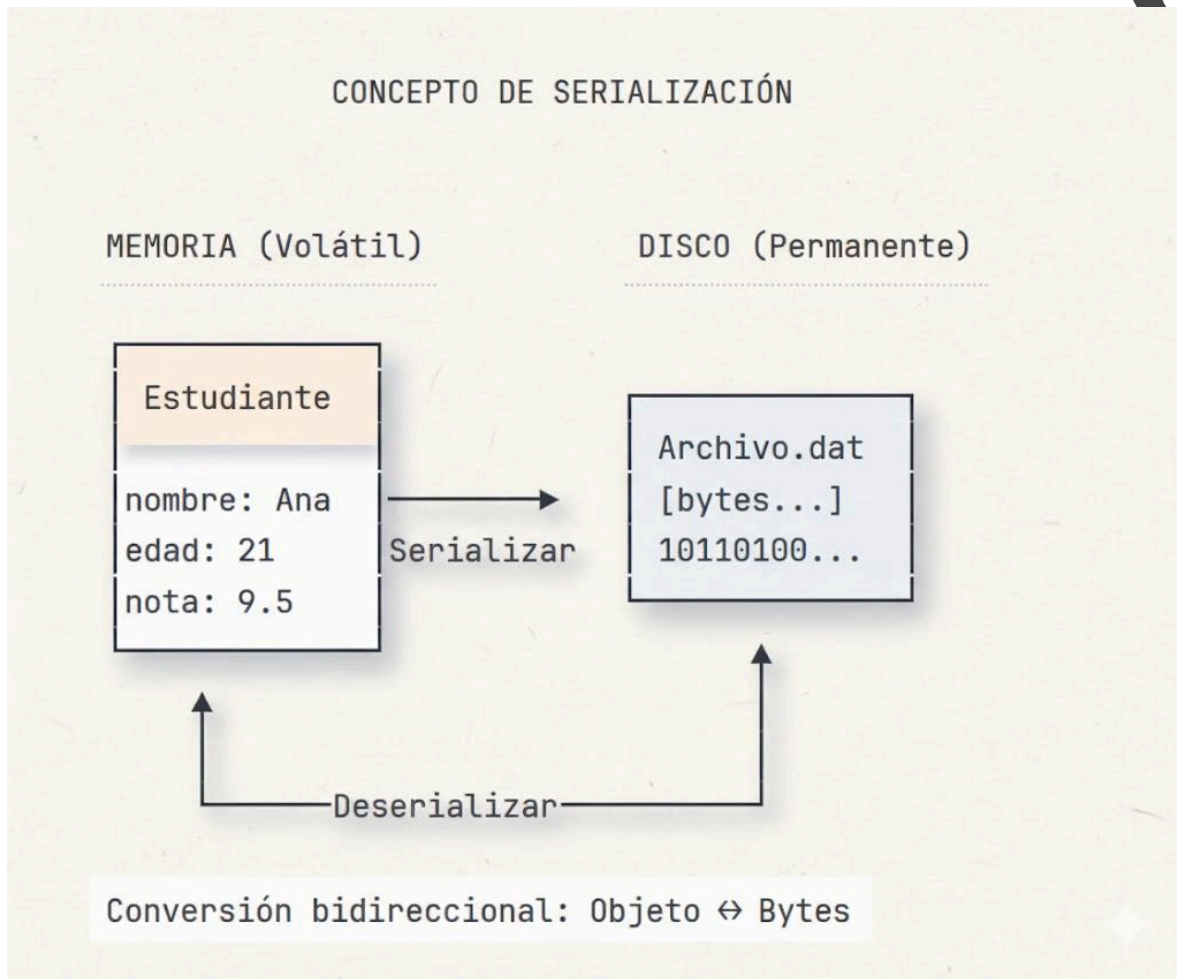
4.2.4.1 SERIALIZACIÓN

¿Qué es la Serialización?

La **serialización** es el proceso de convertir un objeto Java que existe en la memoria en una secuencia de bytes. Esta secuencia puede ser guardada en un archivo del disco duro, enviada a través de una red, o almacenada en una base de datos.

Imaginemos que tenemos un objeto con información de un estudiante en memoria. Cuando serializamos ese objeto, Java lo convierte en una serie de unos y ceros que pueden ser almacenados de forma permanente.





Por qué es importante la Serialización?

2.1 El problema de la volatilidad

Cuando ejecutamos un programa Java, todos los objetos se crean en la memoria RAM. Esta memoria es **volátil**, lo que significa que:

- Se borra cuando cerramos el programa
- Desaparece cuando apagamos la computadora
- Se pierde si hay un error o fallo en el sistema

Sin serialización, cada vez que iniciamos una aplicación, perdemos todo el trabajo previo.

2.2 Aplicaciones prácticas



3. La Interfaz Serializable

Para que un objeto pueda ser serializado, su clase debe implementar la interfaz `java.io.Serializable`. Esta es una **interfaz marcadora**, lo que significa que no tiene métodos que debamos implementar.

3.1 ¿Qué es una interfaz marcadora?

Es una interfaz vacía que sirve como una "etiqueta" o "marca" para indicarle a Java que esta clase tiene permiso para ser serializada. Es como poner una etiqueta que dice "este objeto puede ser guardado".

LA INTERFAZ SERIALIZABLE

```
public interface Serializable {  
    // ¡No tiene métodos!  
}
```

Es una INTERFAZ MARCADORA:

- No tiene métodos para implementar
- Solo indica permiso de serialización
- Java la verifica en tiempo de ejecución

Sin Serializable:

```
NotSerializableException  
El objeto NO puede ser serializado
```

3.2 Ejemplo básico de clase serializable

```
import java.io.Serializable;

public class Estudiante implements Serializable { no usages
    // Identificador de versión (muy importante)
    private static final long serialVersionUID = 1L; no usages

    // Atributos que se guardarán
    private String nombre; 3 usages
    private int edad; 3 usages
    private double promedio; 3 usages

    // Constructor
    public Estudiante(String nombre, int edad, double promedio) { no usages
        this.nombre = nombre;
        this.edad = edad;
        this.promedio = promedio;
    }

    // Getters
    public String getNombre() { no usages
        return nombre;
    }

    public int getEdad() { no usages
        return edad;
    }

    public double getPromedio() { no usages
        return promedio;
    }

    // Método toString para ver el objeto
    @Override
    public String toString() {
        return "Estudiante{" +
            "nombre=" + nombre + '\'' +
            ", edad=" + edad +
            ", promedio=" + promedio +
            '}';
    }
}
```

1. `implements Serializable` - Marca la clase como serializable
2. `serialVersionUID` - Identificador único de versión
3. Los atributos normales se serializan automáticamente
4. Los métodos NO se serializan (solo los datos)

El `serialVersionUID` es un número de identificación único que actúa como una "huella digital" de la clase. Es fundamental para el control de versiones.

4.1 ¿Para qué sirve?

Cuando Java serializa un objeto, guarda el `serialVersionUID` junto con los datos. Más tarde, cuando intenta deserializar (recuperar) el objeto, verifica que el `serialVersionUID` del archivo coincida con el de la clase actual.

FUNCIÓN DEL `serialVersionUID`

GUARDAR (Serialización)

Objeto + `serialVersionUID: 1L` → Archivo

CARGAR (Deserialización)

Archivo (UID: 1L) → Clase (UID: 1L)

✓ Coinciden → ¡Objeto cargado exitosamente!

× No coinciden → `InvalidClassException`

Declaración recomendada

```
private static final long serialVersionUID = 1L;
```

private` - Solo visible dentro de la clase

- `static` - Pertenece a la clase, no a instancias

- `final` - No puede cambiar

- `long` - Tipo de dato numérico largo

- `1L` - El valor inicial (puede ser cualquier número, pero 1L es estándar)

¿Qué pasa si no lo declaramos?

Si no declaramos el `serialVersionUID`, Java lo genera automáticamente basándose en los detalles de la clase. Esto tiene un problema grave: **cualquier cambio mínimo en la clase cambiará el UID automático**, y no podremos leer los archivos guardados anteriormente.

PROBLEMA SIN serialVersionUID EXPLÍCITO

VERSIÓN 1 (Guardamos archivos)

```
class Persona {  
    String nombre;  
    int edad;  
}
```

UID automático: 4567823456L

VERSIÓN 2 (Añadimos un método)

```
class Persona {  
    String nombre;  
    int edad;  
    void saludar() { } ← Método nuevo  
}
```

UID automático: 9876543210L ← ¡Cambió!

RESULTADO: Los archivos viejos ya no se pueden leer
✗ `InvalidClassException`

La palabra clave `transient`

La palabra clave `transient` marca campos que **NO** deben ser serializados. Es útil para información sensible o temporal.

¿Cuándo usar transient?



Ejemplo con transient

```
import java.io.Serializable;

public class Usuario implements Serializable {
    private static final long serialVersionUID = 1L;

    // Estos campos SÍ se serializan
    private String username;
    private String email;

    // Estos campos NO se serializan
    private transient String password; // Sensible
    private transient long sessionId;  // Temporal

    public Usuario(String username, String email, String password) {
        this.username = username;
        this.email = email;
        this.password = password;
        this.sessionId = System.currentTimeMillis();
    }

    @Override
    public String toString() {
        return "Usuario{" +
            "username='" + username + '\'' +
            ", email='" + email + '\'' +
            ", password='" + password + '\'' +
            ", sessionId=" + sessionId +
            '}';
    }
}
```

Comportamiento de campos transient

CAMPOS TRANSIENT EN ACCIÓN

ANTES DE SERIALIZAR:

```
username: "maria123" ✓
email: "maria@email.com" ✓
password: "secreta123" ✓
sessionId: 1699564800 ✓
```

↓ SERIALIZACIÓN

ARCHIVO (lo que se guarda):

```
username: "maria123" ✓ Incluido
email: "maria@email.com" ✓ Incluido
password: [NO INCLUIDO] × transient
sessionId: [NO INCLUIDO] × transient
```

↓ DESERIALIZACIÓN

DESPUÉS DE CARGAR:

```
username: "maria123" ✓ Restaurado
email: "maria@email.com" ✓ Restaurado
password: null ← Valor por defecto
sessionId: 0 ← Valor por defecto
```

6. Campos static

Los campos **static** **NUNCA** se serializan, sin importar si usamos **transient** o no. Esto es porque los campos static pertenecen a la clase, no a instancias individuales.

```
public class Contador implements Serializable {
    private static final long serialVersionUID = 1L;

    // Este campo NO se serializa (es static)
    private static int contadorGlobal = 0;

    // Este campo SÍ se serializa
    private int contadorInstancia;

    public Contador() {
        contadorGlobal++;
        contadorInstancia = contadorGlobal;
    }

    @Override
    public String toString() {
        return "Contador{instancia=" + contadorInstancia +
            ", global=" + contadorGlobal + "}";
    }
}
```

CAMPOS STATIC NO SE SERIALIZAN

Crear 3 objetos:
c1, c2, c3
contadorGlobal = 3 (compartido por todos)

Guardar c2:
Solo se guarda: contadorInstancia = 2

Reiniciar programa y cargar:
contadorInstancia = 2 ✓ Restaurado
contadorGlobal = 0 ← Se reinició

La regla fundamental de la serialización es que **solo se guarda el estado particular (los datos) de la instancia de un objeto.**

- **Campos de Instancia (No `static`):** Se serializan porque definen el estado **único** de ese objeto (ej: `contadorInstancia = 2`). Al deserializar, este valor se **restaura**.
- **Campos `STATIC`:** **NO** se serializan porque pertenecen a la **clase** en su conjunto, y no a una instancia individual. El valor de estos campos (`contadorGlobal = 3`) reside en la memoria de la clase, no en el objeto que se está guardando.

¿Qué sucede al cargar?

Al reiniciar el programa:

1. `contadorInstancia` (campo de instancia) se **restaura** al valor guardado (2).
2. `contadorGlobal` (campo `static`) **no se restaura**. En su lugar, se reinicializa al valor que la clase le da por defecto al cargar el programa (generalmente `0` o el valor inicializado en la clase), demostrando que no fue parte de los datos serializados.

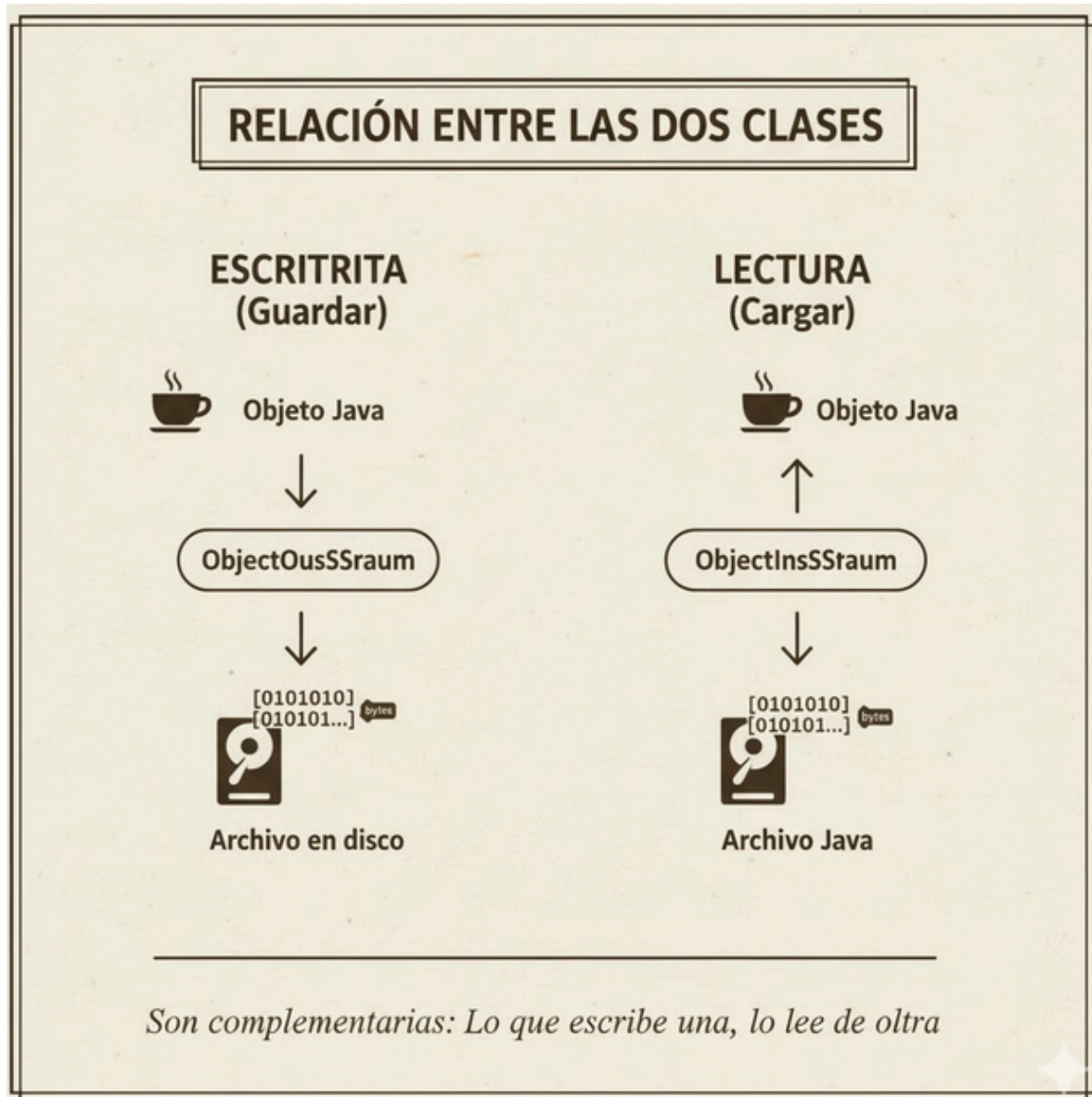
4.2.4.2 CREACIÓN DE OBJETOS `ObjectOutputStream` y `ObjectInputStream`.

¿Qué son `ObjectOutputStream` y `ObjectInputStream`?

Después de entender qué es la serialización, necesitamos conocer las herramientas que Java nos proporciona para llevarla a cabo. Estas herramientas son dos clases fundamentales:

`ObjectOutputStream` - Es la clase responsable de **escribir** (serializar) objetos en un flujo de salida. Convierte objetos Java en bytes y los guarda.

`ObjectInputStream` - Es la clase responsable de **leer** (deserializar) objetos desde un flujo de entrada. Convierte bytes de vuelta en objetos Java.



El concepto de Stream (Flujo)

Un **stream** o flujo es una secuencia continua de datos. Podemos pensar en él como una "tubería" por donde fluyen los datos.

Analogía del mundo real:

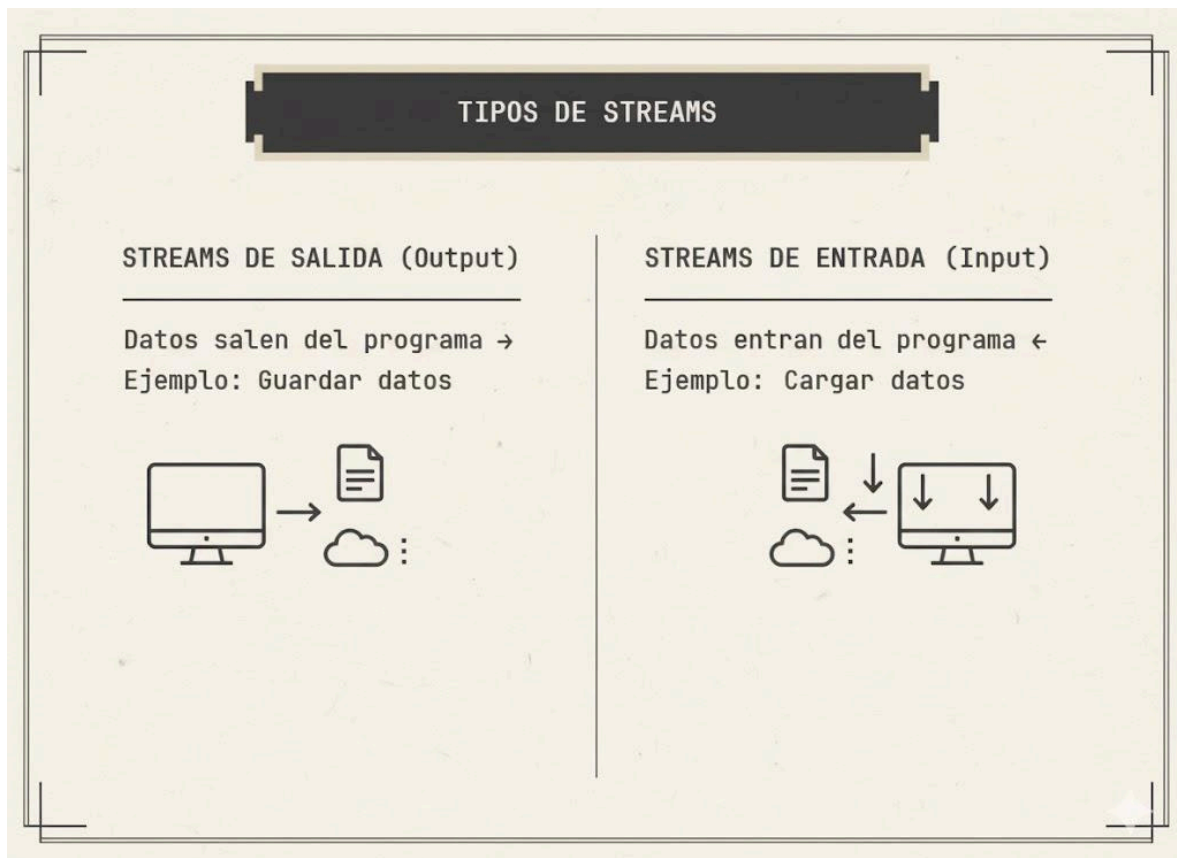
Imagina una manguera de jardín:

- El agua fluye en una dirección

- No puedes "saltar" a la mitad, el agua fluye secuencialmente
- Puedes conectar diferentes secciones de manguera (streams)

De la misma forma, los streams en Java:

- Los datos fluyen en una dirección (entrada o salida)
- Son secuenciales (no aleatorios)
- Se pueden "conectar" o "envolver" unos con otros



ObjectOutputStream - La Clase para Escribir

2.1 ¿Qué hace ObjectOutputStream?

`ObjectOutputStream` es como un "traductor especializado" que toma objetos Java y los convierte en un formato que puede ser guardado. Su trabajo principal es la **serialización activa** - el proceso de escritura.

Funciones principales:

1. Convierte objetos en secuencias de bytes
2. Escribe esos bytes en un destino (archivo, red, etc.)
3. Maneja automáticamente objetos complejos
4. Preserva las relaciones entre objetos
5. Escribe metadatos necesarios para la reconstrucción



Creación de ObjectOutputStream

Para crear un `ObjectOutputStream` necesitamos otro stream que especifique **dónde** se escribirán los bytes. Esto se llama el "stream subyacente" o "underlying stream".

Proceso de creación:

CREACIÓN DE ObjectOutputStream

PASO 1: Crear stream de destino

```
FileOutputStream fos = new FileOutputStream("...");
```

(Define DÓNDE se guardarán los bytes)

 →

PASO 2: Envolver con ObjectOutputStream

 (Añade dos capacidades de serializar objetos)

RESULTADO: Stream listo para escribir objetos



```
ObjectOutputStream oos = new ObjectOutputStream(  
    new  
    FileOutputStream("archivo.dat") );
```

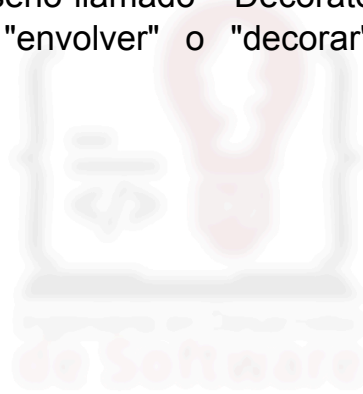
Desglose de la sintaxis:

`new FileOutputStream("archivo.dat")` → Crea/abre el archivo donde se guardarán los bytes

new ObjectOutputStream(...)` → Envuelve el FileOutputStream para añadir funcionalidad de objetos

ObjectOutputStream oos` → Variable que usaremos para escribir objetos

Java usa un patrón de diseño llamado ****Decorator**** para los streams. Esto significa que podemos "envolver" o "decorar" un stream básico con capacidades adicionales.



PATRÓN DECORATOR EN STREAMS

OPCIÓN 1: Básico

`FileOutputStream`

↓ envuelto por

`ObjectOutputStream`

OPCIÓN 2: Con Buffer (más eficiente)

`FileOutputStream`

↓ envuelto por

`BufferedOutputStream` ← Añade buffer en memoria

↓ envuelto por

`ObjectOutputStream`

Ventaja: Cada capa añade funcionalidad

- `FileOutputStream` → Escribe al disco
- `BufferedOutputStream` → Optimiza escrituras
- `ObjectOutputStream` → Serializa objetos

Métodos importantes de `ObjectOutputStream`

`writeObject(Object obj)`

-
- El más importante
 - Serializa cualquier objeto completo
 - Maneja automáticamente objetos complejos
 - Uso: `oos.writeObject(miEstudiante);`

`writeInt(int valor)`

`writeDouble(double valor)`

`writeBoolean(boolean valor)`

`writeLong(long valor)`

- Para tipos primitivos
- Más eficientes que objetos
- Ocupan menos espacio
- Uso: `oos.writeInt(42);`

`writeUTF(String texto)`

- Para cadenas de texto
- Formato UTF-8 (todos los idiomas)
- Incluye longitud automáticamente
- Uso: `oos.writeUTF("Hola");`

`flush()`

- Fuerza escritura inmediata
- Vacía el buffer
- Asegura que datos lleguen al destino

close()

- Cierra el stream
- Libera recursos del sistema
- MUY IMPORTANTE: siempre cerrar

Importancia del cierre de recursos

Cerrar un `OutputStream` es **crítico** por varias razones:

****1. Escritura completa de datos:****

Los datos pueden quedar en buffers de memoria sin escribirse al disco. Cerrar el stream asegura que todo se escriba.

****2. Liberación de recursos:****

Los streams consumen recursos del sistema operativo (descriptores de archivo). Si no los cerramos, podemos agotar estos recursos.

****3. Desbloqueo de archivos:****

Mientras un stream está abierto, el archivo puede quedar bloqueado para otras aplica

¿QUÉ PASA SI NO CERRAMOS EL STREAM?

¿QUÉ PASA SI NO CERRAMOS EL STREAM?

PROBLEMA 1: Datos incompletos

Los últimos datos pueden quedarse en memoria
y nunca escribirse al archivo

Resultado: Archivo corrupto o incompleto

PROBLEMA 2: Fuga de recursos

El sistema operativo mantiene el archivo abierto

Eventualmente: "Too many open files"

PROBLEMA 3: Archivo bloqueado

Otras aplicaciones no pueden acceder al archivo

Posible pérdida de datos si el programa falla

MÉTODOS DE ObjectOutputStream

readObject()

- El más importante
- Lee y deserializa un objeto
- Devuelve Object (requiere casting)
- Puede lanzar ClassNotFoundException
- Uso: `Estudiante est = (Estudiante)ois.readObject();`

`readInt()`

`readDouble()`

`readBoolean()`

`readLong()`

- Para tipos primitivos
- Complemento de los `write...()`
- Deben leerse en el MISMO ORDEN
- Uso: `int edad = ois.readInt();`

`readUTF()`

- Para cadenas de texto
- Complemento de `writeUTF()`
- Lee formato UTF-8
- Uso: `String nombre = ois.readUTF();`

`close()`

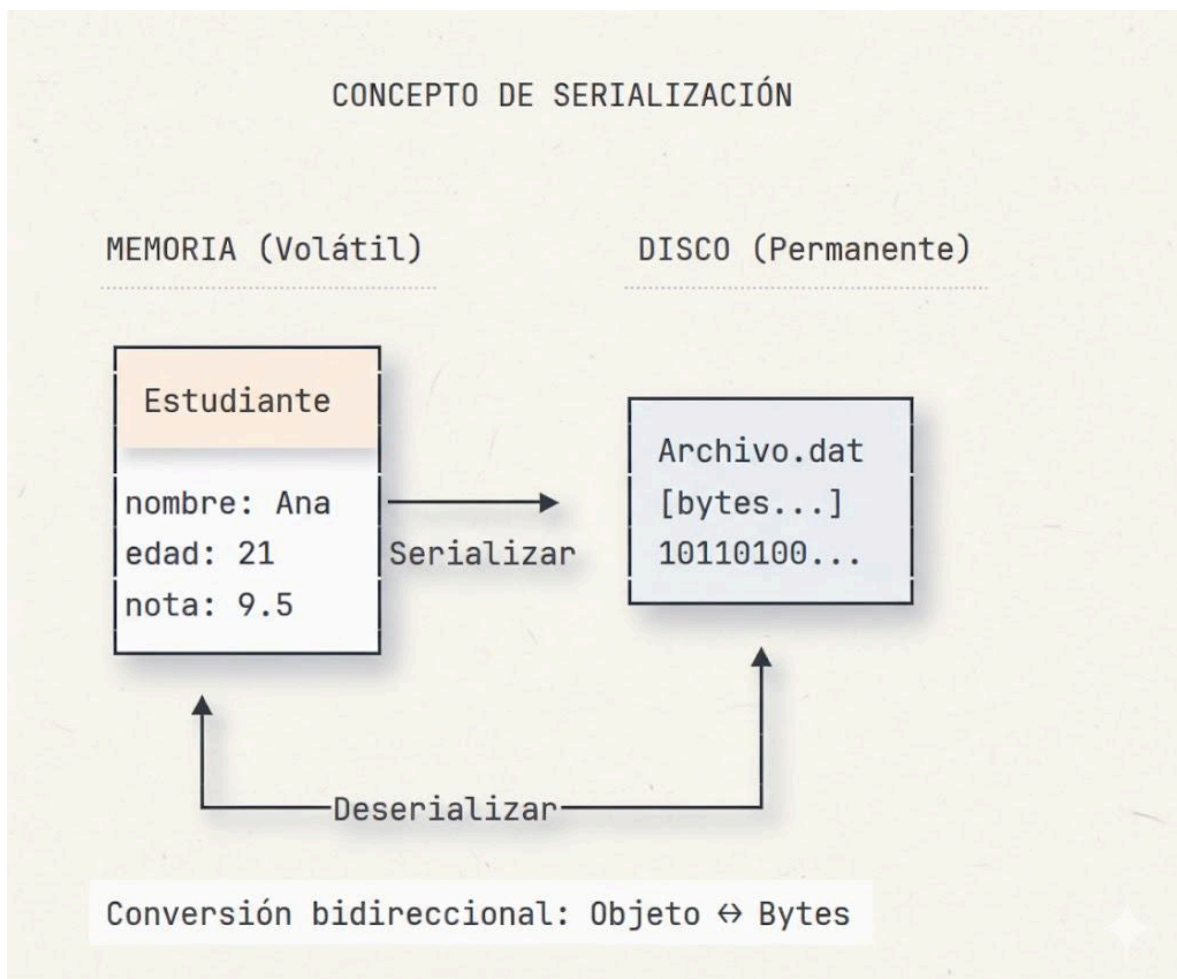
- Cierra el stream
- Libera recursos
- Igual de importante que en escritura

4.2.4.3 DESERIALIZACIÓN DE OBJETOS

Concepto fundamental

La **deserialización** es el proceso inverso a la serialización. Mientras que la serialización convierte objetos en bytes, la deserialización toma esos bytes y los transforma de vuelta en objetos Java completos y funcionales.

Es como "descongelar" un objeto que previamente fue "congelado" mediante la serialización. El objeto recuperado debe ser idéntico al original, con todos sus datos y estado restaurados.



¿Cómo funciona internamente?

Cuando deserializamos un objeto, Java realiza un proceso complejo y fascinante:

Paso 1: Lectura de metadatos Java primero lee la información sobre la clase del objeto: su nombre, paquete, y serialVersionUID.

Paso 2: Verificación de compatibilidad Compara el serialVersionUID del archivo con el de la clase actual. Si no coinciden, lanza una excepción.

Paso 3: Búsqueda de la clase Java busca la definición de la clase en el classpath. Si no la encuentra, lanza ClassNotFoundException.

Paso 4: Creación del objeto Aquí viene algo sorprendente: Java crea una instancia del objeto **sin llamar al constructor**. Usa un mecanismo especial de bajo nivel.

Paso 5: Restauración de campos Lee los valores de cada campo desde los bytes y los asigna directamente al objeto.

Paso 6: Reconstrucción de referencias Si el objeto contiene referencias a otros objetos, Java las reconstruye y las conecta correctamente.

PROCESS INTERNO DE DESERILIZACIÓN

1.  LEER ARCHIVO → **estudiante.dat** → [byts....]
2. EXTRAJER METADATOS
 - Clase: "Estudiante"
 - UID: 1L
 - Campos: nombre, edad, promeido
3. VERIFICAR COMPABIZILIDE
 - UID del achivo == UID de la clase? ✓
4. BUSCAR CLASE
 - ¿Existe "Estudiante" en claspsat? ✓
5. CREAR INSTANCIA
 new Estudiante()
 ← **¡SIN llamar constructor!** ✓
5. RESTARUAR DATOS
 - objeto.nombre = "Ana"
 - objeto.edad = 21
 - objeto.rromeio = 9.5
7. DEVOLVER OBJETO → return objeto → Listo para usar

El Constructor NO se Ejecuta

2.1 Diferencia crucial con la creación normal

Esta es una de las características más importantes y a menudo malentendidas de la deserialización: **el constructor de la clase NO se ejecuta.**

CREACIÓN NORMAL vs DESERIALIZACIÓN

CREACIÓN NORMAL (con new)

```
new Estudiante("Ana", 21, 9.5)
```

↓

1. JVM reserva memoria
2. Constructor se ejecuta ✓
 - Valida parámetros
 - Inicializa campos
 - Ejecuta lógica de negocio
3. Objeto listo

DESERIALIZACIÓN (con readObject)

```
ois.readObject()
```

↓

1. JVM reserva memoria
2. Constructor NO se ejecuta ×
3. Campos se restauran directamente
4. Objeto listo

IMPLICACIÓN: Validaciones del constructor
no se aplican en deserialización

¿Por qué no se llama al constructor?

Esta decisión de diseño de Java tiene varias razones importantes:

Razón 1: Los constructores requieren parámetros ¿Qué valores debería pasar Java al constructor? No tiene forma de saberlo basándose solo en los datos serializados.

Razón 2: Los constructores tienen efectos secundarios Un constructor podría:

- Abrir archivos o conexiones de red
- Registrar el objeto en algún sistema global
- Inicializar recursos del sistema
- Generar IDs o timestamps

Ejecutar estos efectos durante la deserialización causaría problemas impredecibles.

Razón 3: Restauración exacta del estado El objetivo de la deserialización es restaurar el objeto **exactamente como estaba**. Si ejecutáramos el constructor, podríamos sobrescribir los valores guardados con valores de inicialización.

PROBLEMA SI SE EJECUTARA EL CONSTRUCTOR

```
class Contador {  
    private int valor;  
    private long timestamp;  
  
    public Contador() {  
        this.valor = 0; // Inicialización  
        this.timestamp = System.currentTimeMillis();  
    }  
}
```

Guardamos: valor=100, timestamp=1699564800

Si se ejecutara constructor al deserializar:

- valor se resetearía a 0 ✗
- timestamp cambiaría al actual ✗

Solución de Java: NO ejecutar constructor

- valor permanece 100 ✓
- timestamp permanece 1699564800 ✓

Comportamiento de Campos Especiales

3.1 Campos transient después de deserializar

Los campos marcados como `transient` no se serializan, por lo tanto no pueden ser restaurados. Reciben valores por defecto de Java.



VALORES POR DEFECTO DE CAMPOS TRANSIENT

TIPO DE DATO	VALOR POR DEFECTO
byte, short, int	→ 0
long	→ 0L
float	→ 0.0f
double	→ 0.0
boolean	→ false
char	→ '\u0000'
Referencias (Object)	→ null

EJEMPLO PRÁCTICO:

```
class Usuario {
    String username;           // Se guarda
    transient String password; // NO se guarda
    transient int loginCount;  // NO se guarda
}
```

ANTES DE SERIALIZAR:

```
username = "ana123"
password = "secreto"
loginCount = 5
```

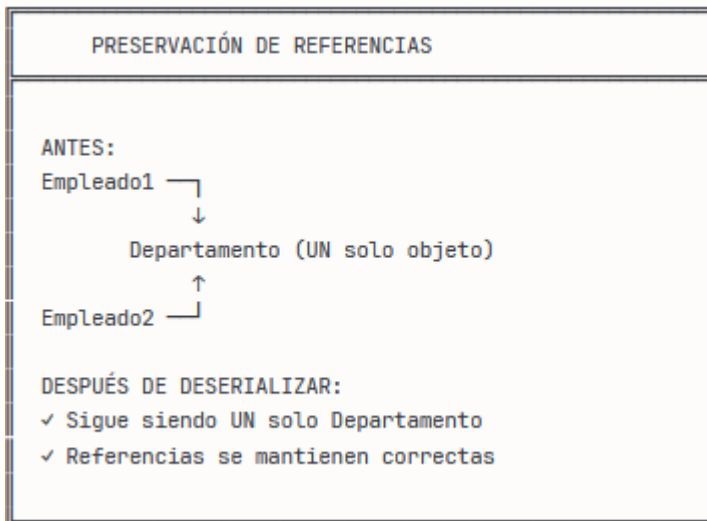
DESPUÉS DE DESERIALIZAR:

```
username = "ana123"    ✓ Restaurado
password = null        ← Valor por defecto
loginCount = 0         ← Valor por defecto
```



Preservación de Referencias entre Objetos

Java mantiene correctamente las relaciones entre objetos al deserializar.



Punto clave: Java serializa cada objeto una sola vez y mantiene las referencias..

Validación Después de Deserializar

Siempre validar los datos después de deserializar, especialmente de fuentes externas.


```
try (ObjectInputStream ois = new ObjectInputStream(  
    new FileInputStream("datos.dat"))) {  
  
    Estudiante est = (Estudiante) ois.readObject();  
  
    // VALIDAR DATOS  
    if (est.getNombre() == null || est.getNombre().isEmpty()) {  
        throw new IllegalStateException("Nombre inválido");  
    }  
    if (est.getEdad() < 0 || est.getEdad() > 150) {  
        throw new IllegalStateException("Edad inválida");  
    }  
  
    // Ahora es seguro usar el objeto  
} catch (IOException | ClassNotFoundException e) {  
    System.err.println("Error: " + e.getMessage());  
}
```

- ✓ Verificar campos no sean null
- ✓ Validar rangos numéricos
- ✓ Comprobar tamaños de colecciones
- ✓ Verificar consistencia entre campos
- ✓ Validar formatos (emails, URLs, etc.)

Seguridad en Deserialización

RIESGOS:

- Ejecución de código malicioso
- Consumo excesivo de recursos
- Inyección de objetos peligrosos

PREVENCIÓN:

- ✓ NUNCA deserializar de fuentes no confiables
- ✓ Validar TODO después de deserializar
- ✓ Considerar JSON para datos externos
- ✓ Limitar tamaño de archivos

Ejemplo Completo Integrado

```

1  import java.io.*;
2  import java.util.ArrayList;
3  import java.util.List;
4
5  class Estudiante implements Serializable { 10 usages
6      private static final long serialVersionUID = 1L; no usages
7
8      private String nombre; 3 usages
9      private int edad; 3 usages
10     private transient String password; 3 usages
11
12     public Estudiante(String nombre, int edad) { 3 usages
13         this.nombre = nombre;
14         this.edad = edad;
15         this.password = "temp123";
16     }
17
18     // readObject personalizado
19     @ private void readObject(ObjectInputStream ois) no usages
20         throws IOException, ClassNotFoundException {
21         ois.defaultReadObject();
22         this.password = null; // Por seguridad
23     }
24
25     public String getNombre() { return nombre; } 1 usage
26     public int getEdad() { return edad; } no usages
27
28     @Override
29     @ public String toString() {
30         return "Estudiante{nombre='" + nombre + "', edad=" + edad +
31             ", password='" + password + "'}";
32     }
33 }
34
35 public class EjemploCompleto {
36
37     // Guardar lista de estudiantes
38     public static void guardar(List<Estudiante> estudiantes, String archivo) { 1 usage
39         try (ObjectOutputStream oos = new ObjectOutputStream(
40             new FileOutputStream(archivo))) {
41             oos.writeObject(estudiantes);
42             System.out.println("✓ Guardados: " + estudiantes.size());
43         } catch (IOException e) {
44             System.err.println("Error al guardar: " + e.getMessage());
45         }
46     }
47
48     // Cargar lista de estudiantes
49     @SuppressWarnings("unchecked") 1 usage
50     @ public static List<Estudiante> cargar(String archivo) {
51         try (ObjectInputStream ois = new ObjectInputStream(
52             new FileInputStream(archivo))) {
53
54             List<Estudiante> lista = (List<Estudiante>) ois.readObject();
55
56             // VALIDAR
57             for (Estudiante est : lista) {

```

```

        if (est.getNombre() == null) {
            throw new IllegalStateException("Datos inválidos");
        }
    }

    System.out.println("✓ Cargados: " + lista.size());
    return lista;
} catch (FileNotFoundException e) {
    System.out.println("Archivo no existe, lista vacía");
    return new ArrayList<>();
} catch (IOException | ClassNotFoundException e) {
    System.err.println("Error al cargar: " + e.getMessage());
    return new ArrayList<>();
}
}

public static void main(String[] args) {
    String archivo = "estudiantes.dat";

    // Crear y guardar
    List<Estudiante> lista = new ArrayList<>();
    lista.add(new Estudiante( nombre: "Ana", edad: 21));
    lista.add(new Estudiante( nombre: "Luis", edad: 22));
    lista.add(new Estudiante( nombre: "María", edad: 20));

    System.out.println("=== GUARDAR ===");
    guardar(lista, archivo);

    // Cargar
    System.out.println("\n=== CARGAR ===");
    List<Estudiante> cargados = cargar(archivo);
    cargados.forEach(System.out::println);
}

```